

אבטחת מערכת web

תיאור החולשה והרקע התאורטי

חולשת **DOM-Based XSS** היא פגיעות אבטחה בצד הלקוח, שבה קוד JavaScript משתמש בקלט הנשלט על ידי תוקף ומזריק אותו ל-DOM בצורה לא בטוחה. בניגוד ל-Stored או Reflected XSS, הפגיעה מתבצעת כולה בדפדפן ללא שינוי בתגובת השרת. הקלט הזדוני מגיע ממקור כגון פרמטרים ב-URL (למשל location.search או location.hash) ומוזרק ל-Sink מסוכן כמו innerHTML, המפרש אותו כ-HTML.

בהיעדר Sanitization או Encoding מתאימים, הדפדפן עשוי להריץ JavaScript זדוני בהקשר האתר, דבר המאפשר לתוקף לבצע פעולות בשם המשתמש ולגשת למידע רגיש. במסגרת הפרויקט נבחנו גם מדיניות **CSP**, מנגנון הגנה בדפדפן המגביל הרצת סקריפטים; נמצא כי יעילותו תלויה בתצורה נכונה, וכי Misconfiguration עלולה לאפשר המשך ניצול הפגיעות.

סביבת העבודה

הפרויקט פותח והורץ בסביבה מקומית (localhost) באמצעות שרת פיתוח מקומי.

כלי העבודה בהם נעשה שימוש:

- עורך קוד: Visual Studio Code
- שרת מקומי: Node.js (באמצעות הפקודה `npm serve`) או Live Server
- דפדפן לבדיקה וניתוח: Google Chrome, תוך שימוש ב-Developer Tools לניתוח ה-DOM ובדיקת התנהגות CSP
- שפות פיתוח: HTML ו-JavaScript בצד הלקוח

מימוש והדגמת המתקפה (POC):

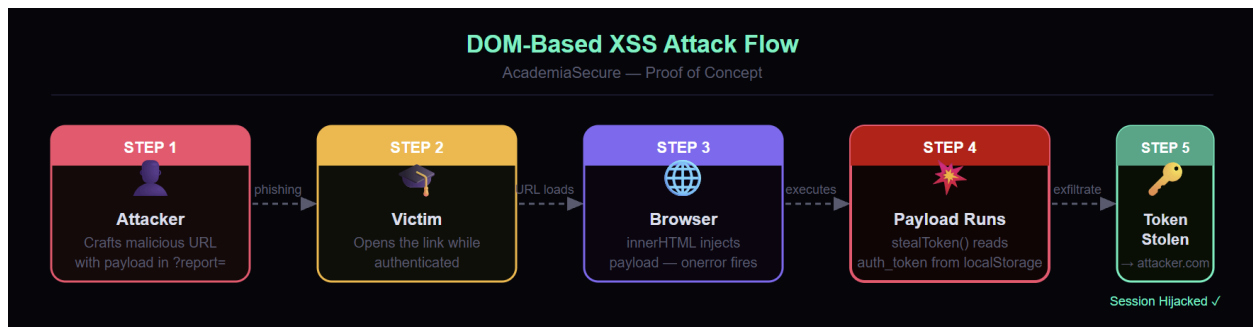
המערכת פגיעה ל-DOM-Based XSS בעמוד ה-Dashboard, כאשר הפרמטר report מה-URL נקרא באמצעות JavaScript ומוזרק ל-DOM דרך innerHTML ללא סינון. התוקף יוצר קישור זדוני הכולל Payload בפרמטר זה; עם פתיחתו, הערך מוזרק לדף, הדפדפן מריץ את הקוד הזדוני, ומתאפשרת גישה לנתונים רגישים כגון auth_token המאוחסן ב-localStorage.

- **לדוגמה :**

- **תוצאה בפועל:**

בגרסה V1 (ללא CSP) ו- CSP (חלש) הקוד הזדוני מורץ והאסימון נגנב.
בגרסה V3 (מחמיר) הרצת JS נחסמת.
בגרסה V4 (שימוש ב-textContent) הפגיעות מתוקנת לחלוטין.

- **דיאגרמה המתארת את שלבי המתקפה:**



מנגנוני הגנה ומניעה:

הפגיעות נובעת משימוש ב-innerHTML להצגת קלט ללא סינון, דבר המאפשר הזרקת קוד זדוני. CSP שימש כמנגנון הגנה משלים המגביל הרצת סקריפטים, אך אינו מתקן את החולשה עצמה.

בבדיקת התצורות נמצא כי ללא CSP או עם CSP (unsafe-inline Weak CSP) ההתקפה הצליחה ונגנב אסימון מ-localStorage. ב-Basic CSP נחסמה הרצת inline JavaScript אך עדיין התאפשרה הזרקת HTML. ב-CSP עם nonce ההתקפה נחסמה לחלוטין.

המסקנה היא שהבעיה העיקרית היא בקוד עצמו: שימוש ב-innerHTML ואחסון אסימון ב-localStorage חושפים את המערכת. לפיכך, יש ליישם Input Validation ו-Output Encoding, להימנע מ-unsafe-inline, לאחסן מידע רגיש באופן מאובטח, ולשלב מספר שכבות הגנה בהתאם לעקרון Defense in Depth ו-Secure by Default.

מקורות:

- <https://ieeexplore.ieee.org/abstract/document/9144421>
- [OWASP DOM Based XSS](#)
- [OWASP Cross-Site Scripting \(XSS\)](#)
- <https://ieeexplore.ieee.org/abstract/document/10725091>